universität wien

MULTIOMICS DATA SCIENCE - PRINCIPAL COMPONENT ANALYSIS

PRINCIPAL COMPONENT ANALYSIS

- A principal component analysis (PCA) is a method for the transformation of **high-dimensional** (numerical) data
- E.g. during a patient examination, measures such as age, height, BMI, body fat percentage, LDL, HDL, and cholesterol values are determined
- These data are so-called **features**, each feature corresponds to one dimension
- A set of features is called an **observation**

DEPENDENCY OF FEATURES

- So my x, y data is therefore two-dimensional, isn‘t it?
- Nope! The y information is a variable which is depedent of x. In math, we call it a projection
- Variable x is therefore a feature and y the **target variable**
- If we want to determine the blood pressure based on age, body height and heart beat frequency, we have three features and one target variable

DEPENDENCY OF FEATURES

- So my x, y data is therefore two-dimensional, isn‘t it?
- Nope! The y information is a variable which is depedent of x. In math, we call it a projection
- Variable x is therefore a feature and y the **target variable**
- If we want to determine the blood pressure based on age, body height and heart beat frequency, we have three features and one target variable
- In a PCA, we use **only features**

FOR WHAT DO WE USE A PCA?

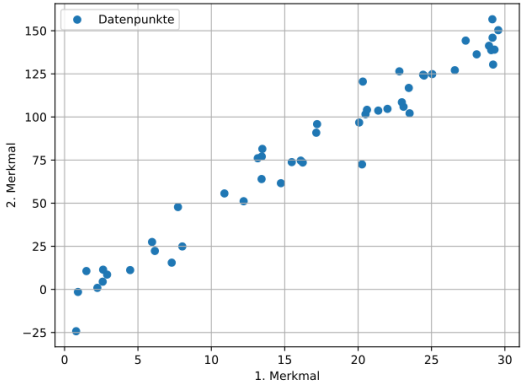
- Data reduction
- Clustering
- Classification
- Feature selection
- Outlier detection
- Recognizing of linear correlations between the features

WHAT IS A PCA ABLE OF?

- **No** data reduction
- **No** clustering
- **No** classification
- **No** feature selection
- **No** outlier detection
- (**No** recognizing of linear correlations between the features)
- Data transformation

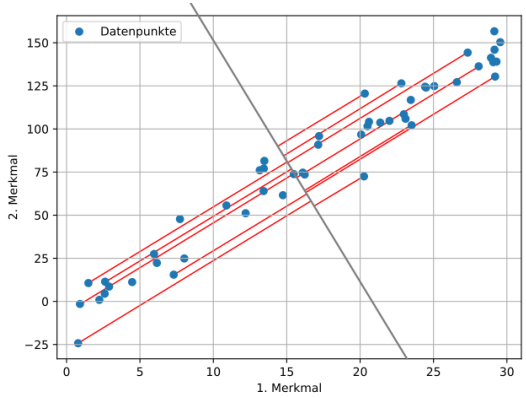
PCA / METHOD

- The input of a PCA is a numerical matrix with *n* features and *m* observations



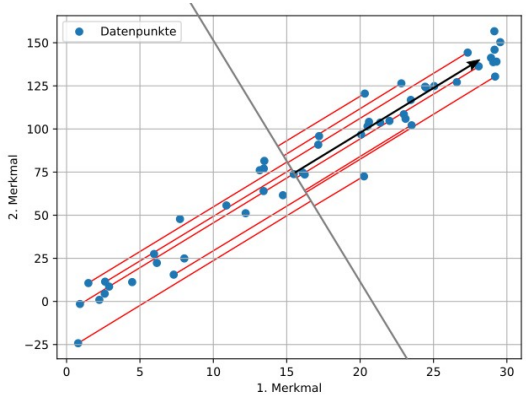
PCA / METHOD

- The input of a PCA is a numerical matrix with *n* features and *m* observations
- In the first step a hyperplane is searched, where all points have the highest summed distance



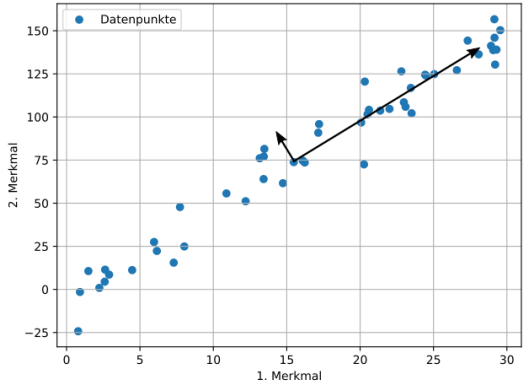
PCA / METHOD

- The input of a PCA is a numerical matrix with *n* features and *m* observations
- In the first step a hyperplane is searched, where all points have the highest summed distance
- The norm vector is the first principal component



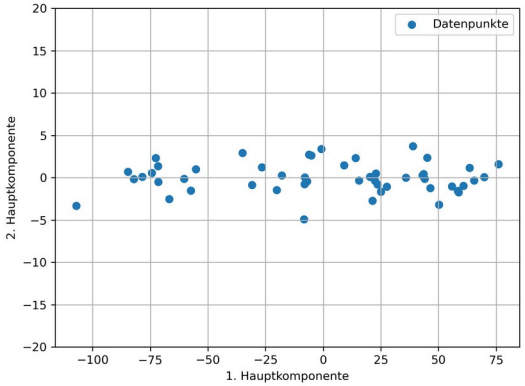
PCA / METHOD

- The input of a PCA is a numerical matrix with *n* features and *m* observations
- In the first step a hyperplane is searched, where all points have the highest summed distance
- The norm vector is the first principal component
- This procedure is repeated where the new norm vector must be orthogonal to all other vectors



PCA / METHOD

- The input of a PCA is a numerical matrix with *n* features and *m* observations
- In the first step a hyperplane is searched, where all points have the highest summed distance
- The norm vector is the first principal component
- This procedure is repeated where the new norm vector must be orthogonal to all other vectors
- At the end, there are as much principal components as features



PCA

- Example on the IRIS dataset:

```
import pandas as pd
iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv')
print(iris)
     sepal_length  sepal_width  petal_length  petal_width    species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
..            ...          ...           ...          ...        ...
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica
```

CORRECT USAGE OF MATRIXES

- An example from analytical chemistry: for *n* samples (observations), we measure the concentrations of *m* proteins (features)

- To use the algorithms and functions provided by the different Python libraries, e.g. for computing the PCA, it is importent that each feature is written **column wise**

- and observations are written **row wise**, such as:

Sample	*Protein 1*	*...*	*Protein n*
Sample 1	1	3	4
...	6	5	7
Sample m	1	2	8

PCA

- A requirement for a PCA is, that the data is normalized, that is for each dimension the mean value is 0 and the variance is 1

- A PCA cannot handle nominal/categorial values, therefore, we need to transform these features into numerical values:

Sample	*Temp.*	*pH value*	*Condition*
Sample1	10	3	Control
...	6	5	Control
Sample m	24	2	Treated

→

Sample	*Temp.*	*pH value*	*Condition Control*	*Condition Treated*
Sample 1	10	3	1	0
...	6	5	1	0
Sample m	24	2	0	1

NORMALIZATION

- Different features have different (norm) ranges, e.g.
  - Puls: 60 – 200 Hz
  - Atmospheric pressure: 100.000 – 101.325 Pa
  - Probabilities: 0 – 1 (without unit)

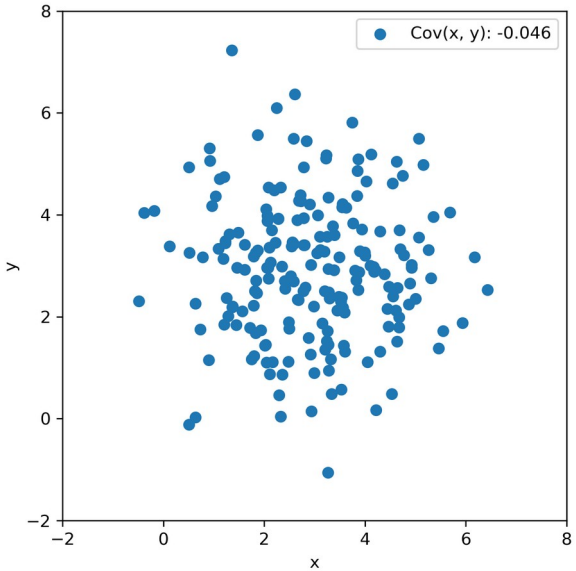
- A PCA works with covariances

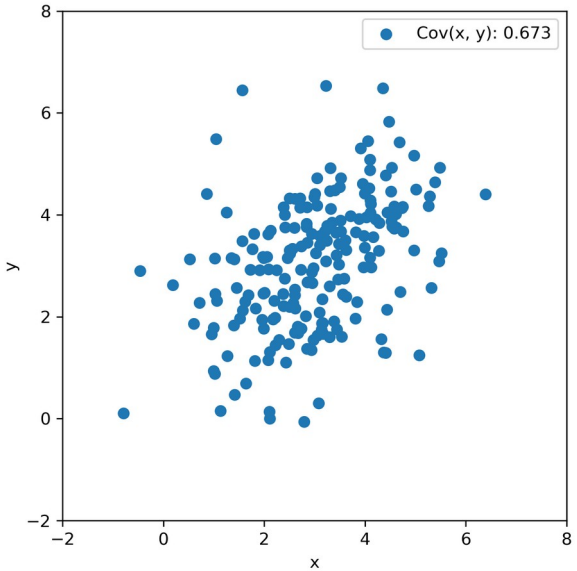
- Without a normalization of the data, the PCA would compare „apples with pears“

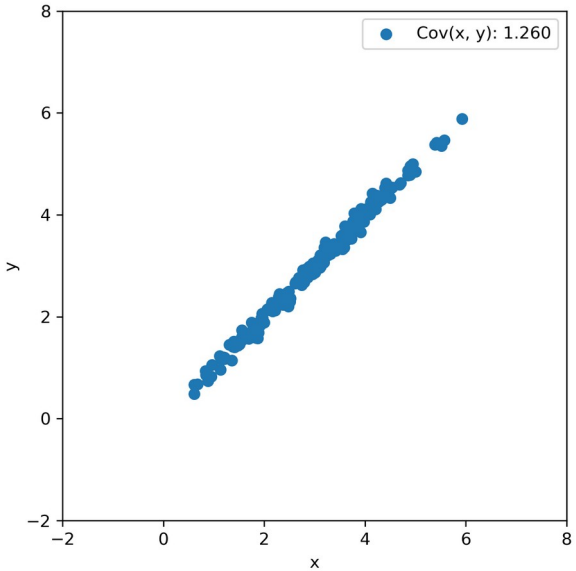
NORMALIZATION / COVARIANCES

- Internally, the PCA pairwisely compares the values of all features with each other and computes linear covariances (measure of dependencies)

- Examples with features of similar variances



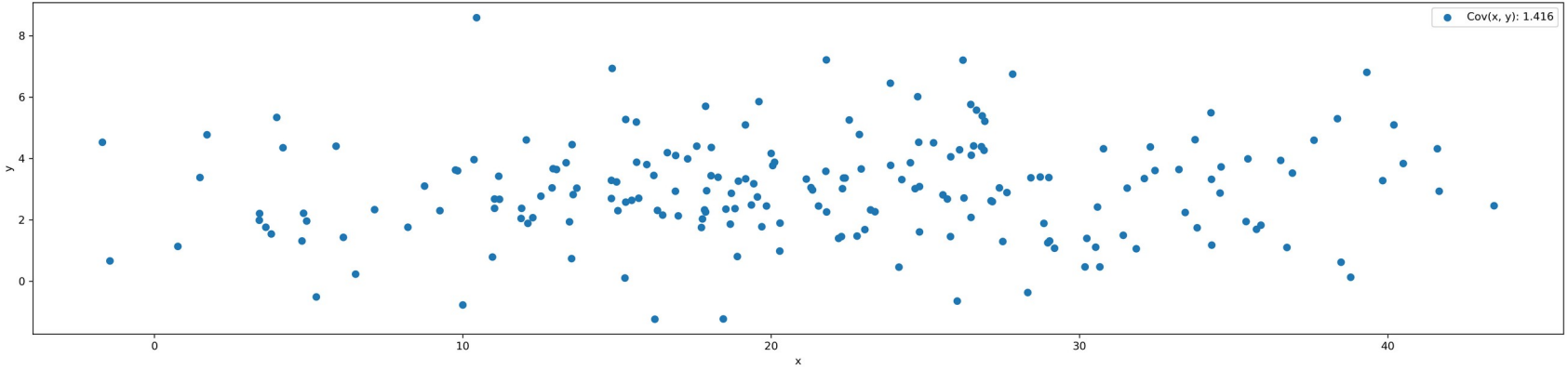




universität wien

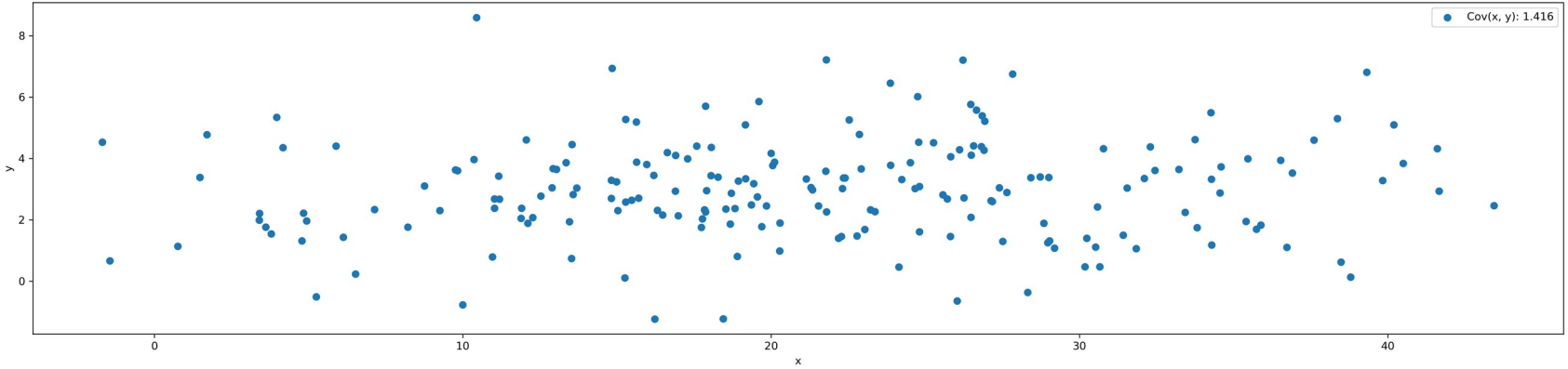
NORMALIZATION / COVARIANCES

- Example of features with different variances, without normalization

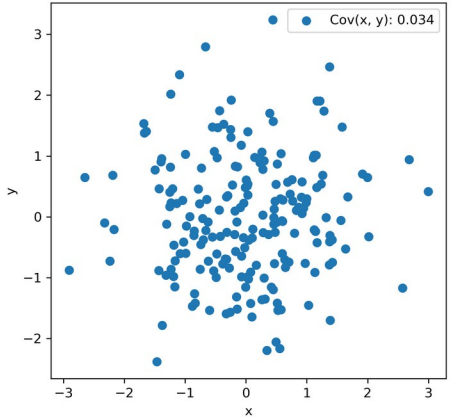


NORMALIZATION / COVARIANCES

- Example of features with different variances, without normalization



- With normalization:



NORMALIZATION

- Manual normalization of the data columns:

```
iris = pd.read_csv("'https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv'")
features = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
x = iris.loc[:, features]# numpy matrix with only numerical values

# Column-wise normalization
for feat in features:
    x.loc[:, feat] = (x[feat] - x[feat].mean()) / x[feat].std()

# Cross check of the normalization
print(x.iloc[:,0].mean(), x.iloc[:,0].std())  # -4.73695e-16 0.9999
```

NORMALIZATION

- Column-wise data normalization with `sklearn`:

```
from sklearn.preprocessing import StandardScaler

iris = pd.read_csv("'https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv'")
features = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
x = iris.loc[:, features]# numpy matrix with only numerical values

# Column-wise normalization
x = StandardScaler().fit_transform(x)

# Cross check of the normalization
print(x.iloc[:,0].mean(), x.iloc[:,0].std())  # -4.73695e-16 0.9999
```

APPLYING THE PCA

- Code for applying the PCA:

```
from sklearn.decomposition import PCA
pca = PCA()                # optional parameter: n_components = 5
components = pca.fit_transform(x)
```

APPLYING THE PCA

- Code for applying the PCA:

```
from sklearn.decomposition import PCA
pca = PCA()                # optional parameter: n_components = 5
components = pca.fit_transform(x)
```

APPLYING THE PCA

- Code for applying the PCA:

```
from sklearn.decomposition import PCA
pca = PCA()                # optional parameter: n_components = 5
components = pca.fit_transform(x)
```

- Ploting of the first two principal components:

```
y = iris["species"]  # target variable
xx, yy = components[:, 0], components[:, 1]

for val in set(y):
    plt.scatter(xx[y == val], yy[y == val])
plt.show()
```

APPLYING THE PCA

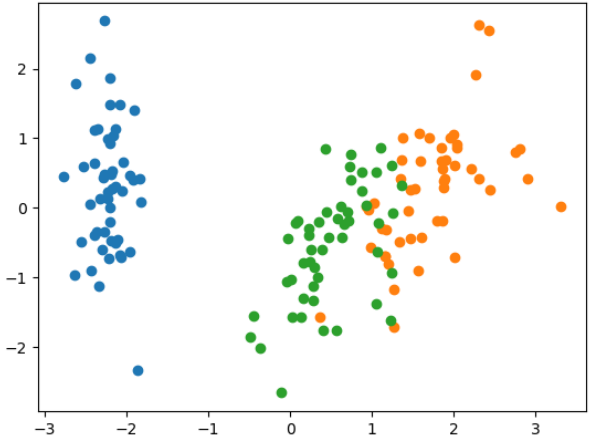
- Code for applying the PCA:

```
from sklearn.decomposition import PCA
pca = PCA()                # optional parameter: n_components = 5
components = pca.fit_transform(x)
```

- Ploting of the first two principal components:

```
y = iris["species"]  # target variable
xx, yy = components[:, 0], components[:, 1]

for val in set(y):
    plt.scatter(xx[y == val], yy[y == val])
plt.show()
```

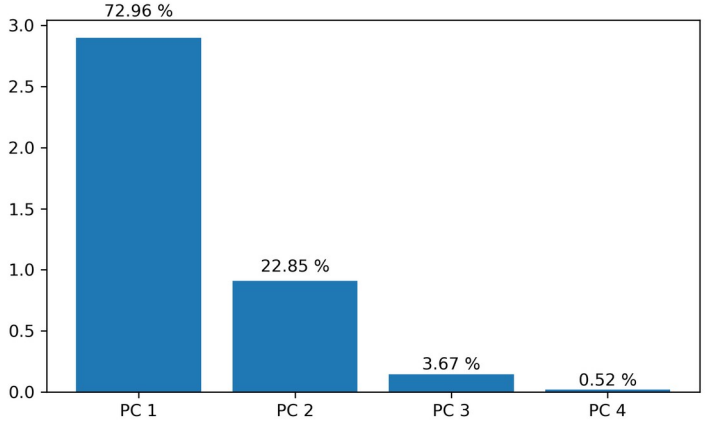


universität wien

VARIANCES OF THE PRINCIPAL COMPONENTS

- The PCA has computed the first principal components with variances in discending order

- By computing and normalizing the variances, we can compare them with each other



VARIANCES OF THE PRINCIPAL COMPONENTS

- The PCA has computed the first principal components with variances in discending order
- By computing and normalizing the variances, we can compare them with each other
- For a dimension reduction, you always should specify the relative variances of the components
- The variances describe how much information is retained in their corresponding components

APPLYING THE PCA

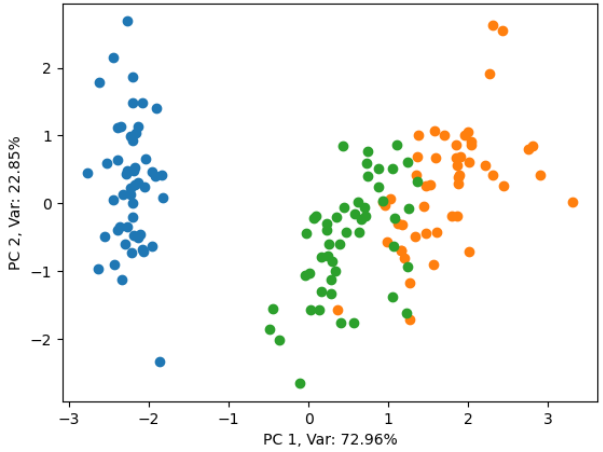
- Code for applying the PCA:

```
from sklearn.decomposition import PCA
pca = PCA()                # optional parameter: n_components = 5
components = pca.fit_transform(x)
```

- Ploting of the first two principal components with variances:

```
xx, yy = components[:, 0], components[:, 1]
vars = pca.explained_variance_ratio_

for val in set(y):
    plt.scatter(xx[y == val], yy[y == val])
plt.xlabel("PC 1, Var: %0.2f%%" % (vars[0] * 100))
plt.ylabel("PC 2, Var: %0.2f%%" % (vars[1] * 100))
plt.show()
```



universität wien

VARIANCES OF THE PRINCIPAL COMPONENTS

- Rule of thumb: for a “good“ PCA, the first 2 – 3 principal components should have at least 80 – 90 % summed variance ratios

- The “best“ result we obtain, when the first principal component already retains 100 % of the data information

- The worst result we obtain, when all dimensions are linearly independent from each other and therefore each component explains only *1 / #Dimensionen* % of the data, that is an equal distribution

DIMENSION REDUCTION

- By applying a PCA, we only obtain a data transform, the number of data points is not changed after applying a PCA, we still have *n* principal components

- We achieve a dimension reduction simply by either keeping a pre-determined number of the first *m* principal components (and the rest we discard), or

- we keep the first *p* principal components, which have a summed variance ratio of at least *x* % retained information (and the rest we discard)

DIMENSION REDUCTION

- By applying a PCA, we only obtain a data transform, the number of data points is not changed after applying a PCA, we still have *n* principal components

- We achieve a dimension reduction simply by either keeping a pre-determined number of the first *m* principal components (and the rest we discard), or

- we keep the first *p* principal components, which have a summed variance ratio of at least *x* % retained information (and the rest we discard)

```
m, x = 3, 0.9
reduced_components = components[:, :m]       # first method

for p in range(1, components.shape[1] + 1):
    if sum(pca.explained_variance_ratio_[ : p]) > x: break
reduced_components = components[:, :p]       # second method
```

RECONSTRUCTION OF THE DATA

- It is possible to reconstruct the original data matrix from the component matrix
- All computational steps of the PCA have to be inverted in reverse order, where lost information during a PCA need to be stored upfront

```
iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
M = np.matrix(iris.loc[:,features])

mu, pca = np.mean(M, axis = 0), PCA()
M_trans = pca.fit_transform(M)
# pca.components_ are Eigen vectors, which are computed during the PCA
M_restored = np.dot(M_trans, pca.components_) + mu
```

RECONSTRUCTION OF THE DATA

- It is possible to reconstruct the original data matrix from the component matrix
- All computational steps of the PCA have to be inverted in reverse order, where lost information during a PCA need to be stored upfront

```
iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
M = np.matrix(iris.loc[:,features])

mu, pca = np.mean(M, axis = 0), PCA()
M_trans = pca.fit_transform(M)
# pca.components_ are Eigen vectors, which are computed during the PCA
M_restored = np.dot(M_trans, pca.components_) + mu
```

OUTLIER DETECTION

- With a PCA, we can detect outliers
- Information get lost during a dimension reduction
- The more a data point (observation) is deviating from the norm (outlier), the more the point is effected by the reduction and looses more information

OUTLIER DETECTION

- With a PCA, we can detect outliers
- Information get lost during a dimension reduction
- The more a data point (observation) is deviating from the norm (outlier), the more the point is effected by the reduction and looses more information
- During a data reconstruction, the reconstructed data point deviates more from its original point
- Here, we can also deterine a threshold from which distance a data point starts to be an outlier
- Determine the *x* % biggest deviations by comparing all deviations

OUTLIER DETECTION

- Python code for outlier filtering:

```
mu, pca = np.mean(M, axis = 0), PCA()
nComp = 2     # reduction size

M_trans = pca.fit_transform(M)
M_restored = np.dot(M_trans[:,:nComp], pca.components_[:nComp,:]) + mu

deviations = scipy.linalg.norm(M - M_restored, axis = 1)   # Euklidean dist.
ths = np.sort(deviations)[int(deviations.shape[0] * 0.95)]     # 5 % outliers

M_filtered = M[deviations < ths]    # filtered matrix without outliers
M_outliers = M[deviations >= ths]   # filtered matrix containing only outliers
```

OUTLIER DETECTION

- Python code for outlier filtering:

```
mu, pca = np.mean(M, axis = 0), PCA()
nComp = 2     # reduction size

M_trans = pca.fit_transform(M)
M_restored = np.dot(M_trans[:,:nComp], pca.components_[:nComp,:]) + mu

deviations = scipy.linalg.norm(M - M_restored, axis = 1)   # Euklidean dist.
ths = np.sort(deviations)[int(deviations.shape[0] * 0.95)]     # 5 % outliers

M_filtered = M[deviations < ths]    # filtered matrix without outliers
M_outliers = M[deviations >= ths]   # filtered matrix containing only outliers
```

- Caution: the filtered matrixes still contain all *m* feature columns but a reduced number of observation rows